Case Study Analysis

TutorMeAI's challenge is not simply adding more features to an AI chat product. The real problem is extending a trusted educational conversation surface into a platform that can safely host third-party applications without losing control over student safety, classroom appropriateness, or conversational continuity. In K-12 settings, a chatbot cannot behave like a consumer app store. Every integration decision changes the trust boundary of the platform. That makes the hardest question not "How do we embed apps?" but "How do we let apps participate in the learning experience without letting them control it?"

The first key problem is trust and safety. Third-party apps are useful precisely because they are created outside the platform, but that also makes them risky. A malicious or poorly designed app could expose student data, show inappropriate content, or manipulate the learning experience in ways schools cannot tolerate. Since TutorMeAI serves children, the platform has to assume apps are untrusted by default. That led to an important ethical and architectural decision: third-party apps must be isolated from the host application with a real browser security boundary. Sandboxed iframes are the best fit because they prevent apps from reading the parent DOM, accessing auth credentials, or inspecting other student data already loaded in the page. Communication must happen only through a tightly controlled `postMessage` contract governed by the Bridge.

The second major problem is state and communication. For TutorMeAI to remain helpful, it must understand what is happening inside an app over time. A chess game, for example, is not a one-time tool call. It is a living interaction where the student may ask for help mid-game, make moves, recover from errors, and later reflect on the outcome. The challenge is preserving that context without allowing the app to write directly into the transcript. The solution is a platform-controlled state layer. The app can send raw events, but the Bridge validates them, stores only approved state, and writes platform-authored summaries into conversation context. This preserves continuity while preventing prompt injection, transcript hijacking, or app-controlled framing.

A third problem is governance. Platform approval and classroom approval are not the same thing. An app may be safe enough to exist on TutorMeAI overall but still inappropriate for a particular teacher, subject, or age group. The best answer is a two-layer approval model: platform admin review first, then teacher or school allowlisting per class. This preserves institutional control while still giving teachers flexibility.

Another important trade-off involves AI ownership. Allowing each app to bring its own model might improve flexibility, but it weakens safety, consistency, and cost control. For this reason, the platform should keep LLM reasoning centralized inside TutorMeAI. Apps contribute UI and structured state, but TutorMeAI owns the model, guardrails, summaries, and final response generation.

Ultimately, the strongest architecture is not a plugin marketplace where third parties are trusted like first-party code. It is a controlled orchestration layer: external apps can register capabilities, render safe UI, and participate in learning workflows, while the platform remains the final authority on data flow, context, permissions, and safety. That approach best matches the ethical requirements of education and gives TutorMeAI a defensible platform advantage rather than a fragile feature expansion.

ChatBridge MVP and Pre-Search Plan

Executive Summary

ChatBridge is a safe orchestration layer for external educational apps inside TutorMeAI. It extends the Chatbox web app with a controlled Bridge that allows third-party apps to register tools, render UI in sandboxed iframes, and communicate through a validated message contract. The platform, not the app, remains the final authority on state, permissions, LLM context, and failure handling.

The MVP planning focus is not broad feature coverage. It is defining a production-quality integration contract that can support multiple app patterns with one generic runtime. The recommended app set is Chess, Weather Dashboard, and Google Classroom Assistant because together they demonstrate complex state, lightweight public integrations, and authenticated educational workflows.

The central design principle is simple: apps may participate in the learning experience, but they may not control it. That principle drives every major architectural choice in this document, including iframe isolation, Bridge-authored context summaries, deny-by-default data sharing, two-layer approval, and platform-owned OAuth.

Project Framing

ChatBridge is a web-native integration layer built on top of the Chatbox web app that allows external educational apps to plug into TutorMeAI safely. The goal is not to let third-party code run inside TutorMeAI directly. The goal is to let third-party apps register tools, render UI inside chat, communicate through a controlled bridge, and preserve student context without ever gaining direct access to the chat transcript, auth tokens, or broader student data.

The platform serves three audiences:

- Platform admin: approves or rejects apps globally.
- Teachers: enable approved apps per class.
- Students: invoke and use approved apps inside chat.

The core product promise is third-party flexibility without giving up classroom safety, platform control, or conversational continuity.

MVP Goals

The planning MVP should optimize for these outcomes:

- Define a secure plugin contract for external educational apps.
- Prove the platform can support multiple app types with one generic bridge runtime.
- Preserve chat continuity even when an app fails.
- Ensure all meaningful app state reaching the LLM is platform-authored, validated, and minimal.
- Support two-level governance: platform approval first, teacher or class allowlisting second.
- Keep the architecture native to the Chatbox web app.

Non-Goals for This Planning Phase

- No coding in the MVP deliverable.
- No full developer portal.
- No marketplace monetization.
- No write-capable Google Classroom actions in v1.
- No arbitrary app-owned LLM usage.
- No direct code plugin system.

Base Stack Assumption

Implementation should stay aligned with the Chatbox web build:

- Frontend: React, TypeScript, and Vite from Chatbox web.
- Platform backend: TypeScript service.
- App embedding: sandboxed iframe.
- App communication: `postMessage`.
- Persistence: database-backed session and policy state.
- LLM orchestration: TutorMeAI-owned model layer.

This is the cleanest path because it extends the existing Chatbox web app rather than building a second system around it.

Users and Permission Layers

There are three separate trust layers:

- Platform admin approval: determines whether an app is safe enough to exist on the platform at all.
- Teacher or class approval: determines whether an approved app is appropriate for a specific classroom.
- Student usage: a student can only access apps that passed both earlier gates.

This creates a strong K-12 governance model: platform safety review protects the ecosystem, teacher controls protect classroom appropriateness, and students never see apps outside their approved class context.

Recommended App Set for MVP

Primary implementation planning set:

- Chess
- Weather Dashboard
- Google Classroom Assistant

Why this set:

- Chess proves high-complexity continuous bidirectional state.
- Weather proves lightweight public app integration.
- Google Classroom proves authenticated external app architecture with educational value.

Planned next-wave expansion:

- AI Story Builder
- Khan Academy or Wolfram lookup
- AI science simulator
- AI drawing canvas

Why Iframe Isolation Wins

The recommended architecture is iframe plus `postMessage`.

Why:

- Browser same-origin policy creates a real security boundary.
- Third-party code cannot read TutorMeAI DOM, memory, tokens, or loaded student data.
- Communication can only happen through bridge-controlled messages.
- The platform explicitly decides what data crosses the boundary.

Rejected alternatives:

- Direct code plugins: too dangerous because third-party code would run in the same JavaScript context and could access everything.
- Web components: acceptable for trusted internal UI composition, not for untrusted third parties. Shadow DOM is not a security boundary.

Conclusion:

- Iframes provide structural isolation.
- `postMessage` provides narrow controlled communication.
- The Bridge becomes the only trusted interpreter of app behavior.

Bridge Architecture

The system should be planned as five major parts:

1. TutorMeAI Web Client
   The Chatbox-based web app students and teachers use. It renders chat, app shelf, active app panel, and follow-up responses.

2. Bridge Runtime
   The controlled layer between TutorMeAI and third-party apps. It validates events, invokes tools, stores app state, and prepares LLM context.

3. Bridge Backend
   Handles app registry, approval state, teacher allowlists, OAuth flows, session persistence, audit logs, and tool orchestration.

4. Third-Party Apps
   Externally hosted educational apps rendered in sandboxed iframes or called through backend endpoints.

5. Policy Layer
   Applies safety rules, data access rules, LLM-safe field policy, approval status, and failure recovery.

Manifest-Driven Platform Contract

The key architectural choice is that the platform should behave generically. New apps should not require new Bridge logic. The manifest defines the app's contract, and the runtime enforces it.

Manifest Fields

Approval fields:

- `app_id`
- `name`
- `version`
- `developer_name`
- `developer_email`
- `support_contact`
- `subject_tags`
- `grade_band`
- `age_rating`
- `content_category`
- `permissions_requested`
- `data_retention_policy`
- `privacy_policy_url`
- `coppa_compliant`
- `ferpa_compliant`
- `content_policy_agreed`
- `review_state`
- `human_review_notes`

Runtime fields:

- `execution_model`
- `url`
- `entry_point`
- `allowed_origins`
- `auth_type`
- `oauth_scopes`
- `tools`
- `tool_result_schema`
- `state_schema`
- `completion_schema`
- `postmessage_schema_version`
- `heartbeat_timeout_ms`
- `ui_capabilities`
- `llm_safe_fields`
- `llm_summary_template`

Highest-leverage manifest fields:

- `tool_name`
- `tool_description`
- `allowed_origins`
- `state_schema`
- `llm_safe_fields`
- `permissions_requested`
- `review_state`

State Model

The Bridge should own the canonical state model. The app never writes to the transcript directly.

There are three kinds of state:

- Volatile UI state: lives inside the iframe only. Cursor positions, animations, hover state. Not stored by the Bridge.
- Interaction state: meaningful current state snapshots like board position, weather location, or current story text. Validated and stored by the Bridge.
- Completion state: important milestones like game over or chapter complete. Validated, persisted, and summarized by the Bridge.

The Bridge should keep an `appContext` object alongside the conversation record. This becomes the only app-facing state object that influences LLM responses.

Rules:

- The app sends raw data.
- The Bridge validates it against schema.
- The Bridge decides whether it is meaningful.
- The Bridge transforms it into platform-authored state.
- The LLM sees only Bridge-authored summaries, never raw app payloads by default.

This is the strongest protection against prompt injection, transcript hijacking, and app-controlled framing.

LLM Context Policy

The LLM context is a teaching surface, not a raw data pipe.

Allowed in context:

- Functional state
- Bridge-authored summaries of meaningful events
- Task context
- Anonymized progress indicators

Blocked by default:

- PII
- Auth credentials
- Raw user-generated content
- Grades and evaluative feedback
- Other students' data
- App internals
- Behavioral or biometric signals

Policy rule:

Every field starts blocked. It only enters context if:

- The manifest declares it LLM-safe.
- The Bridge validates it.
- The Bridge transforms it into a controlled summary.

This should be presented as a deny-by-default safety model.

Tool Invocation Model

The LLM should discover tools from the manifest registry. It should not talk to apps directly.

Flow:

1. Student asks for help or requests an app.
2. TutorMeAI decides whether to route to an approved app.
3. Bridge loads the app iframe if needed.
4. Bridge invokes the selected tool with structured parameters.
5. App responds with validated state or completion events.
6. Bridge updates `appContext`.
7. TutorMeAI answers using Bridge-authored state summary.

This lets app usage feel conversational without allowing the app to control the conversation.

App Discovery UX

Recommended approach: both automatic and explicit.

- Automatic: the model can route to an approved app when intent is clear.
- Explicit: the student can also see a class-approved app shelf and launch apps intentionally.

Why both:

- Automatic routing improves conversational flow.
- Visible app shelf improves trust, discoverability, and user control.

Google Classroom OAuth Architecture

Google Classroom is the authenticated app and should be read-only in v1.

Flow:

1. Student clicks connect Google Classroom.
2. Consent opens in parent window or popup, not iframe.
3. Google redirects back to the Bridge backend.
4. Backend exchanges code for access token and refresh token.
5. Tokens are stored server-side only.
6. On tool invocation, the backend attaches the token.
7. On expiry, refresh happens silently server-side.

Critical rule:

- The iframe never sees OAuth tokens.
- The third-party app never handles token lifecycle directly.
- The Bridge owns credential storage and use.

Failure Handling and Resilience

Core product rule: the chat never dies because an app did.

Failure modes:

- Crash: tear down the iframe, log the event, preserve conversation continuity.
- Hang: detect via timeout or heartbeat, show loading state, degrade gracefully.
- Invalid state: reject payload at schema boundary, keep last valid state, retry once, then continue without the app.

Recovery behavior:

- The user gets clear feedback.
- Chat remains active.
- The LLM is told the app became unavailable.
- The app is treated as optional, not foundational.

Safety and Trust Model

Safety is not just moderation. It is platform design.

Core safety mechanisms:

- Iframe isolation
- Manifest-based permissions
- Schema validation on every event
- Bridge-authored LLM summaries
- Deny-by-default context sharing
- Server-side token handling
- Platform admin review
- Teacher per-class allowlisting
- Audit-friendly stored app session history

This is especially important for K-12 trust. Schools need to know the system minimizes exposure by architecture, not by policy alone.

Recommended Implementation Order After Planning

1. Confirm web-only Chatbox integration path.
2. Define manifest schema and Bridge protocol.
3. Define `appContext` state model and persistence strategy.
4. Design approval workflow and teacher allowlist model.
5. Design iframe embedding and `postMessage` security rules.
6. Design Google Classroom OAuth flow.
7. Design failure handling and fallback UX.
8. Build Chess as the reference implementation.
9. Add Weather as the simple public app.
10. Add Google Classroom as the authenticated read-only app.
11. Add AI Story Builder if time allows.
12. Write developer docs around the manifest contract.

Key Risks

- LLM tool routing ambiguity between multiple apps
- Over-sharing app data into context
- OAuth complexity for external providers
- App lifecycle edge cases across refresh, logout, and resume
- Teachers needing clear permission UX
- External app reliability and trustworthiness

Risk Mitigation

- Strong tool descriptions
- Deny-by-default field policy
- Server-owned auth
- Canonical Bridge session state
- Class-level app controls
- Timeouts, retries, and graceful degradation

Suggested Positioning for Submission

The strongest framing is this: ChatBridge is not a plugin marketplace first. It is a safe orchestration layer for educational apps inside AI chat. The core innovation is the Bridge contract: a generic manifest-driven runtime that lets third-party apps feel native inside TutorMeAI without ever being trusted like first-party code.